

Structural Exhaustion: An Auditable Method for Long-Form LLM-Assisted Proof Development

Guillem Duran-Ballester

August 16, 2026

Abstract

Long mathematical proofs are vulnerable to errors that arise far from their visible consequences. A missing hypothesis, an unjustified independence assumption, or an exhausted quantitative resource may invalidate the intended route only after substantial work has accumulated. We introduce *Structural Exhaustion*, a methodology for developing resilient, LLM-assisted proofs whose logical state remains explicit throughout the argument. The proof is organized as an evolving system of cases, hypotheses, quantitative resources, and closure obligations. Each intermediate claim records its assumptions, its contribution, and the subsequent arguments that consume it.

The central fault-recovery mechanism is branch refinement. When an intermediate argument requires an assumption that has not been established, that assumption becomes a new case distinction. The existing argument is retained on the branch where the assumption holds, while its failure defines a complementary branch requiring an independent analysis. In the motivating hot/cold example, the discovery that an entropy argument required independently realizable configurations preserved the completed hot-branch analysis and exposed a cold branch, which was then developed and closed through separate structural and quantitative arguments.

Structural Exhaustion combines minimal-counterexample reasoning, backward invariant design, explicit resource accounting, first-failure analysis, finite local classification, and exhaustive case routing. These components make local deductions independently auditable and make the completeness of the global argument a concrete mathematical obligation. The resulting proof architecture supports collaboration between language models and human mathematicians while remaining suitable for conventional verification: errors refine the proof’s case structure, valid mathematics is retained, and every newly exposed obstruction receives a precise route toward resolution.

Keywords: LLM-assisted theorem proving; proof planning; structural reasoning; counterexample analysis; auditable proof development.

1 Introduction

1.1 From local competence to long-form proof construction

Modern LLMs can often produce valid individual mathematical moves, while a research proof may require hundreds of mutually dependent moves before the target is reached. Long-form development requires every local result to remain compatible with all earlier exclusions, hypotheses, definitions, and quantitative accounts. These obligations require explicit compositional discipline. Free-form proof development is particularly fragile because an argument can appear globally convincing

while leaving a difficult case, boundary correction, or unstated assumption unresolved. Structural Exhaustion arose from attempting to continue proof construction beyond the scale at which ordinary conversational proof development remained dependable.

1.2 Method and contributions

The method treats a hypothetical counterexample as an adversarial object that must continually pay for avoiding the target. Every admitted dichotomy assigns the positive side to a quantitative or structural argument and the negative side to a new obstruction that can itself be analyzed. A failed lemma or structural guess can reveal the next invariant, local witness, label, carrier, or residual class. Its counterexample becomes input to the subsequent branch. The proof develops by repeatedly transforming ignorance about the counterexample into explicit restrictions on what the counterexample is still allowed to be. Structural Exhaustion combines familiar proof mechanisms into a method for constructing and auditing long arguments. Its novelty lies in organizing those mechanisms into a systematic construction process in which every branch, failure, account, and residual has an explicit mathematical role. The resulting proofs can be long because the method expands case handling and bookkeeping that expert mathematical prose often leaves implicit. Structural Exhaustion provides a practical methodology for generating invariants, budgets, branch orders, residuals, and closure routes from the resources already present in a problem. It explains why this methodology is especially compatible with the kinds of structured mathematical work that LLMs can perform effectively. The methodology was generalized retrospectively from the Erdős–Gyárfás development, whose construction preceded the reference manual. The accompanying repository, reference manual, proof manuscripts, diagrams, and audit records provide the detailed mathematical artifacts behind the compact presentation in this paper.

The present article gives a methodological account of decomposition, maintenance, and audit in large proofs. Each theorem-level statement inherits the verification status established in its companion manuscript.

2 Design assumptions for LLM assistance

2.1 Structural reasoning and invariants

LLMs can reason about structural consequences of assumptions: what a graph, profile, decomposition, or counterexample must look like if a given property holds or fails. Structural Exhaustion turns this capability into a systematic operation by promoting properties with declared downstream consequences into explicit branch invariants. The method asks which local or quantitative property would make the next available tool applicable. This question produces an operational branch predicate directly from the tool’s hypotheses. The positive and negative sides of that property then become separate sources of mathematical progress.

2.2 Bookkeeping and explicit representations

LLMs can manipulate large collections of named quantities, cases, labels, constraints, and cross-references when those objects are explicitly represented. Structural Exhaustion turns such bookkeeping into mathematics by expressing global obstruction through demands, payers, capacities, deficits, surpluses, and other named currencies. The bookkeeping is generative: an overloaded payer, exhausted capacity, or unbalanced budget forces new structure. Rigorous accounting therefore converts the ability to maintain many small relationships into a mechanism for discovering the next

proof branch. LLMs can read and generate structured textual objects such as tables, dependency diagrams, case matrices, and finite-state descriptions. Structural Exhaustion makes extensive use of exactly these representations because a long proof contains many relationships that are awkward to preserve reliably in compressed prose. Branch tables expose missing cases, dependency tables expose unsupported conclusions, and ledgers expose double counting or unavailable resources. The diagrammatic representation is consequently part of the reasoning process. It records branch coverage and dependencies in a form that can be checked directly.

2.3 Standard techniques and counterexample analysis

LLMs have broad familiarity with standard mathematical moves such as minimal-counterexample arguments, pigeonhole principles, charging, switching, first failure, compactness, replacement, and finite enumeration. Structural Exhaustion builds large arguments by composing these familiar mechanisms. Novel work is concentrated in the instance-specific interfaces, invariants, and closure lemmas. Complexity is moved from the primitive moves into their composition: a long proof can be assembled from individually recognizable local arguments. The CT1–CT17 library is a vocabulary for recurring combinations of standard mechanisms. Each application carries explicit instance-specific hypotheses. LLMs can assist in attacking a proposed claim and finding a configuration in which it fails. Structural Exhaustion treats that tendency as an asset because a counterexample to an intermediate claim exposes additional structure that the surviving adversarial object must possess. A failed proposed normalization can reveal an exceptional local configuration, a failed estimate can identify a deficient account, and a failed equivalence can expose a distinguishing context. Thus each proposed claim is evaluated through two questions: “Can we prove this?” and “What structural information follows from its failure?”

2.4 The resulting division of labor

The model generates, expands, criticizes, classifies, and reorganizes local mathematics within declared interfaces. Global conclusions require the verified composition of those local outputs. Within the workflow, LLMs maintain state, propose case splits, fill ledgers, generate local tests, and draft named obligations. Human review and mathematical verification determine the validity of the proposed local lemmas, routing contracts, computations, and closures. Structural Exhaustion therefore attempts to exploit what LLMs do well while placing their characteristic long-form failure modes inside an explicit mathematical discipline.

3 Extracting structure from the counterexample

Consider a class $\mathcal{X}$ of mathematical objects, a baseline predicate H , and a target predicate T . The intended theorem has the form

$$\forall X \in \mathcal{X}, \quad H(X) \implies T(X).$$

Structural Exhaustion begins with an object X satisfying $H(X) \wedge \neg T(X)$, usually chosen minimal with respect to a well-founded order. The method then constructs a finite or well-founded tree of certified state transitions. Each leaf either yields the target, contradicts minimality, violates a capacity bound, reduces to a checked finite family, or exports a typed residual to a declared consumer.

3.1 Start from the adversary

For a theorem asserting that every object satisfying (H) contains a target (T), the method begins by assuming a counterexample satisfying $H \wedge \neg T$. Whenever possible, the counterexample is chosen minimal under a well-founded order so that deletion, replacement, or compression immediately becomes available as a contradiction mechanism. Minimality converts many otherwise harmless local modifications into structural restrictions on the counterexample.

3.2 Replace the global target by local tests

The next step is to identify bounded or otherwise tractable tests whose realization implies or detects the desired target. Target avoidance is encoded as the absence of an explicit family of local events. This encoding replaces the undifferentiated predicate $\neg T$ by testable structural data. Every absent local event becomes information that later branches are allowed to exploit.

3.3 Resource inventory and backward design

Before inventing branches, list the mathematical resources already available and record exactly what each one can prove. Typical resources include minimality, target algebra, imported theorems, charging schemes, finite label sets, exact types, computational certificates, and branches already known to close. A proposed proof step is admissible when each possible outcome can eventually be paid for from this inventory. The generative question is: “What hypothesis would allow an available tool to make progress?” Its answer supplies the next candidate invariant. Structural Exhaustion therefore designs invariants backwards from the estimates, charging arguments, or closure mechanisms one hopes to apply. The organizing question is: “What can the present proof resources already pay for, and which additional hypothesis would make that payment valid?”

3.4 The both-sides test

A proposed invariant P is admissible when both P and $\neg P$ have productive mathematical consequences. The positive side must feed an account, estimate, or closure mechanism. The negative side must force a bounded residual, first-failure witness, structural obstruction, or another named route. The actual admission rule asks what the positive side pays, what the negative side forces, whether the predicate is recorded, and which consumer receives the residual. An admissible invariant assigns a mathematical consequence to each side of its dichotomy. A predicate whose failure has no declared consequence remains outside the branch state.

3.5 Accumulate an invariant ladder

Each successful dichotomy places another certified restriction on the surviving counterexample. Later estimates are used only after the hypotheses they require have been established along the surviving branch. The resulting main case is constructed gradually by routing every alternative to its accumulated properties through a declared branch.

Formally, if B_i denotes the state after the first i admitted moves, then the structural record is monotone:

$$H(B_i) \subseteq H(B_{i+1}), \quad E(B_i) \subseteq E(B_{i+1}), \quad I(B_i) \subseteq I(B_{i+1}),$$

except where an explicitly proved transport lemma replaces one representation by an equivalent one. This monotonicity ensures that every later estimate has established hypotheses and that every local modification transports the earlier exclusions it needs.

4 Structural accounting

4.1 Demands and payers

Once the proof identifies structures that must occur in bulk, those structures are represented as demands that must be assigned to limited resources or payers. A charging scheme specifies the demands, the eligible payers, a canonical assignment rule, and the maximum capacity of each payer. The global inequality follows from locally justified assignments, multiplicity bounds, and capacities.

A convenient abstract form is a demand set D , a payer set P , and a canonical assignment $\pi : D \rightarrow P$. If each payer p has certified capacity $c(p)$, then a non-overloaded assignment must satisfy

$$|D| = \sum_{p \in P} |\pi^{-1}(p)| \leq \sum_{p \in P} c(p).$$

Every symbol in this inequality has a local proof obligation: demands must be distinct, the assignment must be total, ties must be resolved consistently, and each capacity estimate must be valid under the current branch state. If total demand exceeds available capacity, some payer or label class is overloaded. The overloaded class often yields a homogeneous family, matching, star, chain, or repeated local configuration. This extraction strengthens a numerical contradiction into a structural conclusion. Quantitative pressure is therefore converted back into structural information.

4.2 Budgets and currencies

Different resources such as deficiency, surplus, entropy, rank, boundary mass, or concentration are treated as distinct currencies unless a proved interface converts one into another. Keeping currencies separate prevents a proof from quietly paying the same obstruction twice. Competing demand families either need disjoint payers or an explicitly recorded split of the common supply.

4.3 Backward rate computation

Numerical coefficients are selected from the gap between what the proof can supply and what the counterexample is forced to demand. The substantive mathematical fact is the strict gap between the two rates. Any coefficient in the resulting admissible interval supports the same closure. Thresholds can then be obtained by solving the corresponding supply-demand equality.

For example, suppose a family of size n forces at least $an - b$ demands and the available payers have total capacity at most $cn + d$, where $a > c$. Any counterexample must then satisfy $(a - c)n \leq b + d$. Thus all objects above the threshold $(b + d)/(a - c)$ close by overload, while the objects below it form a bounded residual family. The coefficients are outputs of local estimates; the algebra only identifies the range in which those estimates can close the branch.

4.4 Moves $\times$ budgets

Every structural move—deletion, replacement, peeling, charge transfer, or handoff—is checked against every quantitative account it may affect. A permitted move preserves each recorded deficit and satisfies the declared monotonicity bound on the adversary’s supply. When a move breaks monotonicity, the failure identifies the missing boundary term, surplus correction, or incompatible budget. The resulting moves $\times$ budgets table and its slack accounting form a design audit.

5 Turning structural repetition into closure

5.1 Finite labels and exact external types

Finite labels encode exactly the bounded classification required by the proof. They omit raw structural data outside the declared downstream interfaces. Many objects carrying the same label then create the homogeneous subfamilies on which standard combinatorial mechanisms become applicable. For replacement arguments, coarse similarity is insufficient because two pieces may behave differently when embedded into different outside contexts. The method therefore refines pieces by their target-relevant external responses until equality of type is strong enough to support replacement. A shorter representative with the same relevant external behavior contradicts minimality.

If K is a piece with boundary ∂K , let $\mathcal{C}_{\partial K}$ denote the allowed outside contexts with the same interface. Two pieces K_1, K_2 have the same target-relative external type when

$$T(C \oplus K_1) \iff T(C \oplus K_2) \quad \text{for every } C \in \mathcal{C}_{\partial K}.$$

This equivalence is deliberately target-relative. Recording all internal data would defeat compression, whereas recording too little would make replacement unsound. The proof must establish that its chosen finite descriptor decides the displayed equivalence in the declared class of contexts.

5.2 Exchange and first failure

When two comparable local structures are exchanged or compared, three fundamental outcomes recur. A hit means that the comparison directly realizes the target. A defect means some context distinguishes the two structures and thereby supplies new structural information. A compression means no relevant context distinguishes them, allowing the larger structure to be replaced by a smaller one. This hit/defect/compression trichotomy turns local repetition into an exhaustible proof mechanism. When a desired property fails along a sequence, chain, scale hierarchy, or iterative construction, the proof selects its first failure. This choice localizes the obstruction while retaining the valid prefix. Everything before the first failure satisfies the desired condition, while the failure itself supplies a bounded local witness. This converts an unbounded negative condition into a finite structural object that can be charged, labelled, or routed.

5.3 Repetition, pumping, and descent

Long sequences of finitely typed states eventually repeat, and repeated exact behavior creates an opportunity for compression, periodicity, or a distinguishing witness. Finite-state repetition thus turns long structures into bounded proof obligations. Some residuals are eliminated iteratively by removing one certified unit of obstruction at a time. Each iteration must re-establish the required branch state and decrease a well-founded measure. Iteration is admissible when restoration and termination have been proved.

6 A reusable closure-tactic vocabulary

The reference manual names seventeen closure tactics, CT1–CT17. They cover local target tests, minimality and replacement, external-type compression, charging, local-to-global bookkeeping, active-dormant splitting, exchange, finite-state pumping, overload, default refinement, localization, peeling, tiered charging, aggregate closure, rank forcing, whole-object exact types, and target thickening. The list is a vocabulary of contracts. Every use must prove the contract’s instance-specific inputs.

6.1 Why a tactic vocabulary is useful

The same structural patterns recur across many branches, so giving them stable names reduces the need to rediscover their interfaces repeatedly. CT1–CT17 package recurring operations such as target testing, replacement, charging, exchange, overload, peeling, rank forcing, exact typing, and target thickening.

6.2 Tactic contracts and instance obligations

A tactic contract states what data a move consumes, what obligations must be proved, and which closure or residual types it may return. The contract records obligations whose validity must be established in the mathematical instance under consideration. Instance-specific mathematics remains responsible for constructing the definitions, lemmas, bounds, and witnesses required by the tactic.

6.3 Mechanism versus strategy

The tactic library describes mechanisms, whereas the strategy layer decides when to invoke them, how to choose parameters, and how to compose them. This separation places every problem-specific assumption and strategic choice in the strategy layer.

7 Operational workflow

7.1 The branch state

A branch is represented by its complete accumulated context. This representation gives precise content to phrases such as “the remaining difficult case.” The context is represented by

$$B = (H_0, \preceq, E, I, R, V, Q, A),$$

whose coordinates contain hypotheses, progress information, exclusions, invariants, residual data, vocabulary, queued payloads, and audit records. Later arguments may consume only information that has actually been established and recorded in the current branch.

The coordinates have distinct roles. H_0 stores standing hypotheses; $\preceq$ records the well-founded progress measure; E and I store certified exclusions and positive invariants; R contains the current residual data; V fixes the vocabulary in which that data is expressed; Q is the queue of typed payloads awaiting consumers; and A is the audit record. A transition is admissible only when it states which coordinates it reads, which it changes, and why the untouched coordinates remain valid.

7.2 Operational cycle

Starting from the current branch state and residual queue, one iteration of Structural Exhaustion consists of the following six stages.

- 1. Propose.** Form an invariant, case split, budget, label, local test, exchange, or candidate lemma that may advance the current branch.
- 2. Admit.** Test the proposal for productive outcomes, measurability, available resources, and a declared route for every surviving case. Imported global results enter as theorem obligations; bookkeeping moves use only resources already recorded in the branch state.

3. **Select.** Choose an admitted move whose prerequisites and parameters are available or can be synthesized from the current state.
4. **Execute.** Prove, review, or compute the local mathematical obligations before allowing the move to change the proof state.
5. **Route.** Close successful outcomes and emit every surviving obstruction as a typed residual with a declared downstream consumer. Similarity-based routes require an explicit transport or reduction lemma.
6. **Record.** Update the branch tree, invariant and exclusion records, ledgers, dependency information, and residual queue.

The cycle is therefore $\text{state+queue} \rightarrow \text{propose} \rightarrow \text{admit} \rightarrow \text{select} \rightarrow \text{execute} \rightarrow \text{route} \rightarrow \text{record}$.

Writing $\text{Step}(B, q)$ for this operation, the central safety condition is leaf totality. Every returned outcome belongs either to a declared closure class or to the domain of another registered step. Consequently, the architecture detects every branch lacking a consumer before a reader assesses the harder local mathematics.

8 Reliability controls for LLM-assisted development

The architecture addresses recurrent long-form failure modes through explicit admission and audit requirements.

Branch omission.

Every leaf is either closed or retained as a typed residual, exposing any unhandled case.

Unsupported global estimates.

Every candidate inequality is decomposed into named demands, assignments, multiplicity bounds, and budget comparisons before admission.

Equivalent restatement.

An auxiliary lemma must shrink, charge, route, or otherwise structure the original difficulty.

Untyped remainders.

Failure of the current tools is converted into a named residual carrying the data needed by its consumer.

Double counting.

Canonical assignments, separated currencies, and slack records control independent uses of a common resource.

Unverified acceptance.

A proposed statement enters the proof state only after its argument or certificate has discharged the declared obligations.

Uncontrolled extrapolation.

New global results are isolated as explicit theorem or research obligations; routine branches use established local mechanisms.

9 Defects, counterexamples, and repair

When a proposed lemma fails, its failure conditions are preserved as structural data. The resulting counterexample can reveal a missing term, boundary case, incorrect interface, insufficient label, or new residual pattern. Repairs follow a four-level hierarchy.

1. **Numerical repair** recomputes constants, finite enumerations, precision bounds, or slack within the existing quantitative layer.
2. **Structural repair** corrects a hypothesis, invariant, route, type, or account within the existing proof vocabulary.
3. **Pattern-level repair** promotes a repeatedly surviving residual to a reusable proof mechanism with a declared interface.
4. **Language extension** adds the descriptors and transitions needed to represent an obstruction outside the current vocabulary.

Explicit residual states also provide graceful degradation. Withdrawing one closure leaves a precise weaker statement identifying the remaining counterexample class and localizes the effect of the defect.

10 Audit artifacts

10.1 Diagrams, tables, and ledgers

Large branch systems are represented by dependency diagrams whose nodes correspond to mathematical statements, dichotomies, or closures. The diagrams present branch coverage directly to human readers and models. Branch tables make the outcomes of a dichotomy explicit, while requirement tables record what each lemma consumes. Reverse indexes expose which downstream arguments depend on a changed invariant. Quantitative ledgers record actual proof resources, while exclusion and invariant ledgers record the structural facts accumulated along a surviving branch. The resulting bookkeeping is sufficiently rich that later mathematical deductions can be generated by reading the state of the ledger itself.

10.2 The role of explicit volume

Structural Exhaustion expands information that conventional proofs often leave implicit. The additional length exposes information needed for local LLM reasoning, explicit review, and later repair.

11 The Erdős–Gyárfás case study and the origin of Structural Exhaustion

The Erdős–Gyárfás conjecture asks whether every graph of minimum degree at least three contains a cycle whose length is a power of two. Earlier work has established the conjecture under restrictions on induced paths (Gao and Shan, 2021; Hu and Shen, 2023; Hegde et al., 2024), while classical results on even cycle lengths supply part of the surrounding extremal context (Bondy and Simonovits, 1974; Sudakov and Verstraëte, 2008). The development discussed here is the companion candidate manuscript (Duran-Ballester, 2026), together with its audit materials and current review record. The

Erdős–Gyárfás development preceded the Structural Exhaustion reference manual, which generalized the recurring moves required during its construction. The case study supplied the recurring patterns from which the methodology was subsequently abstracted.

11.1 What the development demonstrates

The proof combines target arithmetic, minimality, induced-path structure, charging, entropy, rank, local exchanges, finite tables, and residual ledgers across a long companion manuscript. Its proof architecture preserves successive structural restrictions along a long minimal-counterexample spine and composes them through local closure mechanisms. Its residual ledgers and branch diagrams enforce preservation of the invariants accumulated before each surviving obstruction.

Concretely, the graph-theoretic state transports minimum-degree information, induced-path restrictions, admissible cycle-length windows, local response types, and several separate charge accounts. A branch that rules out one cycle-producing configuration returns the graph together with the certified absence of that configuration. The next stage therefore receives a strictly stronger structural state from every non-closing predecessor.

11.2 Methodological abstraction

Repeated use of local tests, invariant ladders, charging, first failure, exact types, exchange, compression, and residual routing suggested that these mechanisms could be separated from the particular combinatorial problem. Structural Exhaustion turns that proof-development experience into a reusable methodology.

12 The nonlinear PDE case study

The PDE case studies instantiate the same proof-engineering architecture with analytic state variables. A graph label may be finite and literal, whereas a PDE profile can carry scale, center, gauge, compactness, concentration, and ancestry data. The common structure lies in the interfaces: a state records established facts, a first failure creates a localized witness, and every unresolved alternative transports the data that generated it. A substantially different domain tests the structural philosophy on compactness, concentration, pressure gauges, descendant profiles, and analytic case distinctions. The resulting Navier–Stokes development spans three interacting manuscripts with an extensive system of local analytic routes.

12.1 Type-I setup

The setup manuscript converts a hypothetical local Type-I singularity into a normalized ancient Sereg state carrying fixed concentration, ancestry, and normalization data. Its 39-node dependency architecture then routes that state through explicit Liouville alternatives and a residual branch.

Normalization is a state transition that transports the subject of the argument into a specified ancient solution. The ancient solution inherits quantitative data from the hypothesized singularity, and subsequent nodes use the transported properties. If a Liouville route fails, the emitted residual therefore contains both the failed criterion and the normalization data needed by the closure manuscript. The residual manuscript takes the exact generated state space left by the setup paper and refines its remaining case into a sequence of explicitly named residual classes. The manuscript retains the original raw generated hull and ancestry data throughout the refinement of the unresolved object.

This retention requirement is analytically substantive. Compactness can produce many related limits. Conclusions about a descendant profile transfer to the original generated state through explicit ancestry maps and gauge transports, which specify exactly which properties survive passage between the objects.

12.2 Type-II development

The Type-II manuscript expands the same structural philosophy into an even larger local state-space analysis involving repaired gauges, multibubbles, cascades, pressure routing, scale collapse, cost criteria, and terminal residuals. Its proof-dependency diagram is one twelve-part graph with numbered nodes through 135 and a separate first-failure closure layer. This example illustrates what Structural Exhaustion looks like when the local vocabulary becomes highly domain-specific while the general principles of explicit branches, residuals, first failures, and closure accounting remain recognizable.

At this scale the dependency diagram functions as an index to the proof. Its node number identifies a mathematical statement, its incoming edges identify the hypotheses available at that statement, and its outgoing edges enumerate the possible conclusions. The separate first-failure layer converts a negative statement about an entire cascade into the scale and witness at which the relevant property breaks.

12.3 Evidence supplied by the case studies

The methodological significance of these artifacts lies in their explicit organization of dependencies, branches, residuals, and audit obligations. They demonstrate that LLM-assisted construction can be organized into large, highly structured mathematical developments extending beyond isolated short proofs. The detailed mathematical correctness of the headline theorem claims remains a separate review question from the proof-development methodology.

13 Relationship to existing mathematics and AI theorem proving

13.1 Classical proof techniques

Structural Exhaustion draws on established mathematical traditions such as minimal counterexamples, reducibility, discharging, finite-state reasoning, switching, and replacement.

Reducibility and discharging in the proofs of the Four-Colour Theorem provide a prominent precedent for replacing a short global argument by a large family of explicit local obligations (Appel and Haken, 1977; Appel et al., 1977; Robertson et al., 1997). Modern accounts of discharging likewise emphasize that local charge conservation and capacity estimates must be specified before the global contradiction follows (Cranston and West, 2017). The primitive techniques are therefore mostly familiar; the proposed contribution is their systematic organization into an LLM-compatible long-form construction methodology.

Exact external types also have classical neighbours in context equivalence and finite-state theory (Myhill, 1957; Nerode, 1958). In graph algorithms, protrusion replacement gives a more specialized comparison: bounded-interface pieces are classified by their behaviour under gluing and replaced by representatives (Bodlaender et al., 2009; Kim et al., 2012). Structural Exhaustion uses the same broad idea as a proof-development interface, with the target predicate determining which responses the type must preserve.

13.2 Formal proof and proof planning

Formal systems provide stronger final guarantees, while proof-planning systems similarly represent methods through explicit preconditions and effects. Structural Exhaustion occupies an earlier stage: organizing the discovery, drafting, branching, and repair of the informal mathematics before or alongside final formal verification.

This positioning makes the architecture compatible with formalization. A typed residual can become a theorem statement, a tactic contract can become an interface in a proof assistant, and a finite certificate can be checked by a small verified program. Formalization then checks the local claims and their composition; the methodology supplies an explicit decomposition from which those formal goals can be generated.

13.3 AI theorem proving

Much AI theorem-proving work focuses on solving individual formal goals, competition problems, or formalizing existing source mathematics.

Representative systems use language models for autoformalization, for a draft-sketch-prove pipeline, or for retrieval-augmented proof search in Lean (Wu et al., 2022; Jiang et al., 2022; Yang et al., 2023). These directions improve the translation and discharge of formal goals. Structural Exhaustion addresses the complementary problem of determining the interacting goals, branch states, and residual interfaces of a long developing argument. Structural Exhaustion targets the construction and maintenance of large mathematical arguments composed from many interacting local decisions. The contribution lies in organizing dependency graphs, formal interfaces, and textbook tactics into a complete proof-development process.

14 Scope and limitations

14.1 Verification requirements

Structural Exhaustion makes obligations and residuals explicit. Mathematical validity additionally requires a proof or certificate for every local lemma. Human review and, where available, formal verification establish the mathematical claims from the supplied arguments and certificates.

Architectural completeness and mathematical validity impose independent requirements. Local validity requires correct edge lemmas; global coverage requires an exhaustive branch graph. Review establishes both properties. A successful compilation checks the syntactic portion, while formalization can certify the encoded mathematical claims and their composition.

14.2 Empirical status of the design assumptions

The architecture rests on practical design assumptions derived from development experience. Their applicability depends on the model, domain, prompting regime, and available verification tools.

14.3 Extensibility of the proof language

Some mathematical problems require additional descriptors, global witnesses, analytic quantities, or proof mechanisms beyond the current tactic vocabulary. Such situations extend the language with problem-specific data, transitions, and closure contracts.

14.4 Cost of explicitness

Explicit structural bookkeeping produces additional text, tables, and local obligations relative to a compressed specialist proof. This explicit information supports long-form LLM-assisted development and detailed auditing.

15 Conclusion

Structural Exhaustion begins from a simple inversion of ordinary proof search: every unsuccessful attempt to force the target is analyzed for new information about the counterexample. LLMs contribute structural reasoning, counterexample search, textbook-method recall, diagrammatic organization, and extensive bookkeeping throughout this process. Rigorous branch and resource accounting turns those capabilities from disconnected local suggestions into a cumulative proof-development process. The resulting methodology extracts structural information from counterexamples, while bookkeeping assigns every acquired fact to a downstream argument or closure. The large combinatorial and nonlinear-PDE developments motivate the claim that this approach can support LLM-assisted mathematical construction at a scale far beyond isolated short proofs. The repository, reference manual, proof corpus, and audit artifacts provide the detailed material needed for readers to inspect, use, criticize, and extend the methodology. The method may be compressed into the following progression:

productive two-sided branches $\implies$ typed structural obstructions,
typed structural obstructions $\implies$ recorded cumulative pressure,
recorded cumulative pressure $\implies$ one of the declared closure classes.

Iteration continues until every branch has a declared terminal certificate or an explicitly stated residual obligation.

Acknowledgments and AI-use disclosure

Large language models were used as drafting and proof-engineering aids. The author reviewed the resulting text and remains responsible for every mathematical and methodological claim. The evidentiary basis consists of the arguments, computations, and certificates supplied in the manuscripts.

References

- Kenneth Appel and Wolfgang Haken. Every planar map is four colorable. I. Discharging. *Illinois Journal of Mathematics*, 21(3):429–490, 1977.
- Kenneth Appel, Wolfgang Haken, and John Koch. Every planar map is four colorable. II. Reducibility. *Illinois Journal of Mathematics*, 21(3):491–567, 1977.
- Hans L. Bodlaender, Fedor V. Fomin, Daniel Lokshantov, Eelko Penninkx, Saket Saurabh, and Dimitrios M. Thilikos. (Meta) Kernelization, 2009.
- J. A. Bondy and M. Simonovits. Cycles of even length in graphs. *Journal of Combinatorial Theory, Series B*, 16(2):97–105, 1974.

- Daniel W. Cranston and Douglas B. West. An introduction to the discharging method via graph coloring. *Discrete Mathematics*, 340(4):766–793, 2017.
- Guillem Duran-Ballester. A structural exhaustion proof of the erdős–gyárfás power-of-two cycle problem: An LLM-assisted minimal-counterexample analysis. Candidate companion manuscript under public review, distributed with the Structural Exhaustion reference manual, 2026.
- Yuping Gao and Songling Shan. Erdős–gyárfás conjecture for P_8 -free graphs, 2021.
- Anand Shripad Hegde, R. B. Sandeep, and P. Shashank. Erdős–gyárfás conjecture on graphs without long induced paths, 2024.
- Zhiquan Hu and Changlong Shen. Erdős–gyárfás conjecture for P_{10} -free graphs, 2023.
- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2022.
- Eun Jung Kim, Alexander Langer, Christophe Paul, Felix Reidl, Peter Rossmanith, Ignasi Sau, and Somnath Sikdar. Linear kernels and single-exponential algorithms via protrusion decompositions, 2012.
- John Myhill. Finite automata and the representation of events. Technical Report WADC Technical Report 57-624, Wright Air Development Center, 1957.
- Anil Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- Neil Robertson, Daniel P. Sanders, Paul Seymour, and Robin Thomas. The four-colour theorem. *Journal of Combinatorial Theory, Series B*, 70(1):2–44, 1997.
- Benny Sudakov and Jacques Verstraëte. Cycle lengths in sparse graphs. *Combinatorica*, 28(3):357–372, 2008.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models, 2022.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models, 2023.